VisualLogic

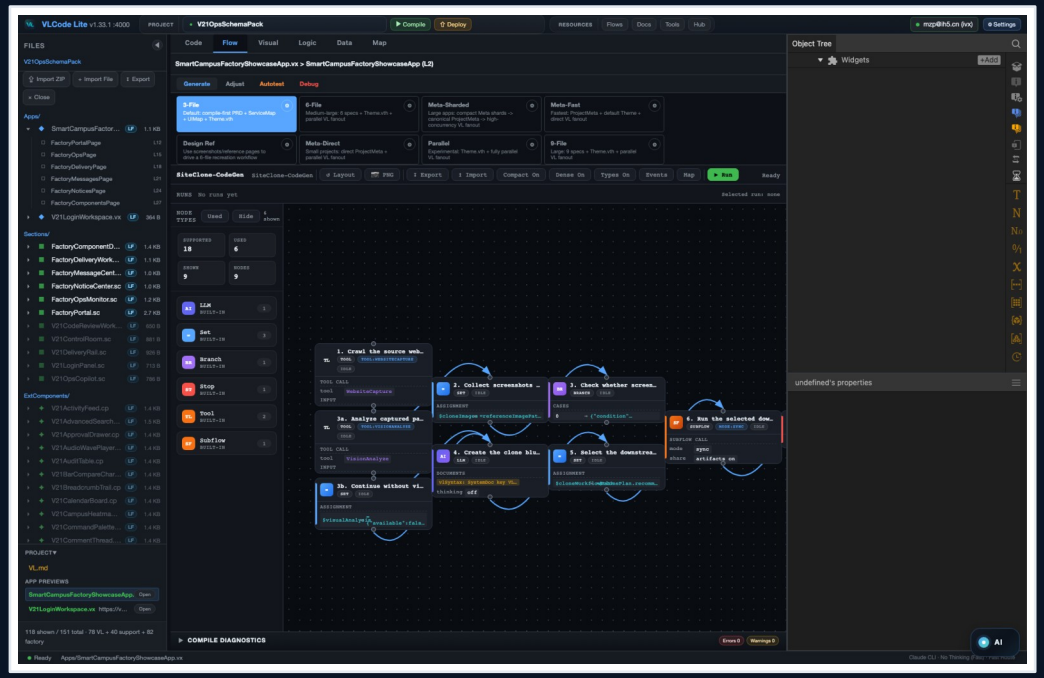
LOCAL AGENT APP STUDIO · AGENT OS

Agents you can install, pause & audit.

A local, runtime-first studio for agents, tools, approval flows and automation — each one a single, portable .vflow.

Open source · Local-first, not SaaS

github.com/VisualLogic-AI/VLCode-Lite-agent-os



Every Agent App runs as a DAG you can pause and resume.

01 TWO PRODUCTS, ONE FOUNDATION

VL Agent produces. VLCode consumes. .vflow connects.

VL Agent

Agent App Studio + local Agent OS. Authors / governs / validates the automation.

.vflow

VLCode

Enterprise app dev base. Imports the .vflow to generate / adjust / orchestrate apps.

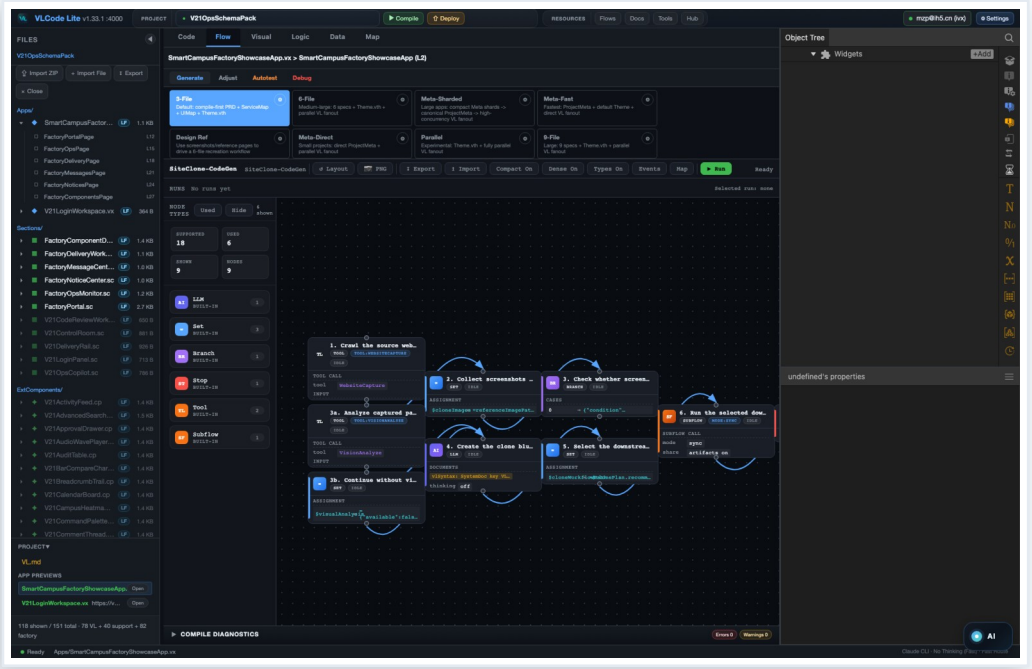
Same VL · same DAG engine · same Resource Center · one account · one download.

02 THE SHARED SPINE

Both products tame AI inside the same DAG.

AI is brilliant at a point; it can't hold a line. A DAG is the bridge from point intelligence to process intelligence — creativity in the nodes, determinism in the edges. VLCode tames AI into enterprise apps; VL Agent tames AI into governed, auditable automation. Same engine underneath.

> ***Creativity in the nodes. Determinism in the edges.***



A run unfolds into a finite, acyclic, auditable trace.

03 WHY A PROMPT ISN'T ENOUGH

A reliable agent must be installed, paused, resumed, audited, governed.

A chat agent lives inside the prompt. A real automation cannot: it must survive restarts, wait on humans, record every step, and answer “what happened and why” weeks later. That needs a runtime — not a longer prompt. VL Agent is that runtime, on your own machine.

> ***A chat agent needs a prompt. A dependable agent needs a runtime.***

04 WHAT VL AGENT IS

A local Agent App Studio — and a local Agent OS.

> ***“A local Agent OS centered on workspace execution, workflow runtime, control-plane persistence, and resource governance. VL is the first deep domain pack, not the system boundary.”***

A desktop workbench for building runtime-first AI applications on your own machine: agents, tools, approval flows, automations, and process apps.

05 WHAT YOU BUILD

An Agent App is a single, portable .vflow.

An entry workflow + a top-level app block. Install · start · pause · resume · audit.

Manifest → Workflow Engine executes → Control Plane persists → VL renders.

Approval Flow Runtime

Durable background approval — requester / approver cards + SLA reminders.

Goal Convergence console

Inspect goal contracts, task frontiers, scorecards, mainline, rollback.

Website Clone Studio

DAG-first authorized site exploration → VL clone.

Paper-to-Agent-App

Turn a paper into a runnable Agent App.

Shipped in the box — grounded examples, not roadmap.

06 THE LOCAL AGENT OS UNDERNEATH

Every run recorded. Every resource reviewed.

SQLite control plane

Records every run, task, artifact and review — any run is replayable and auditable.

150+ workflows out of the box

From 3-/6-/9-file VL code generation to multi-actor team delivery, approval runtimes and data factories.

Resource Center governance

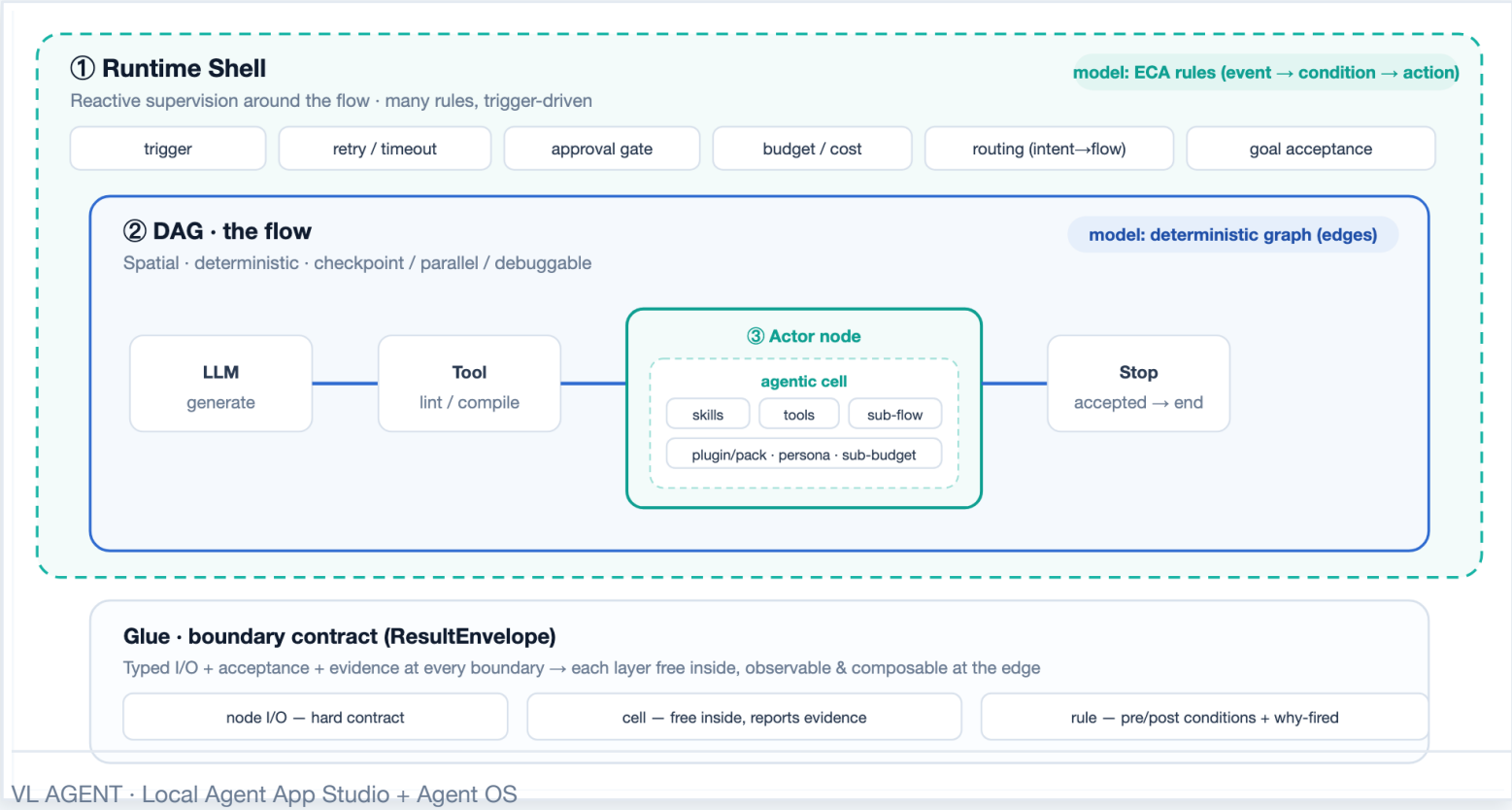
Review-gated lifecycle for components / flows / docs / tools / skills / packs. Resources pass through review before they ship.

Built-in vlflow CLI — dry-run-validates any workflow without mutating it; closure expansion embeds subflows so a .vflow ships self-contained.

07 THE RUNTIME

Three runtimes. One system.

Each layer uses the computational model that fits it — never one nested inside another.



① Shell · rules

Event→condition→action around the flow: trigger, retry, approval, budget, routing, goal acceptance.

② DAG · deterministic

Explicit edges, checkpoint, parallel, node breakpoints. The stable, reproducible spine.

③ Cell · exploratory

An Actor node mounts skills + tools + sub-flows and explores freely — held by a typed contract.

08 DAG VS SKILLS

When to draw a graph. When to hand it to the agent.

Dimension	Skills · agent loop	DAG · workflow
Control flow	Model decides — non-deterministic	Edges & branches fixed — deterministic
Debugging	Hard — no nodes, no breakpoints	Breakpoint / pause / rerun / checkpoint
Parallelism	Mostly sequential	Real Fork / Loop fan-out
Acceptance	Self-declared “done” (soft)	Explicit gates + Score / Review
Observability	Weak	Run ledger + artifact lineage
Authoring cost	Low — a few sentences	Higher — lay out the graph
Best for	Open-ended, judgment, one-off	Repeatable, parallel, audited

Where new logic goes: **repeatable → DAG** · **judgment → skill in an Actor node** · **reactive → shell rule**

09 THE CONNECTOR

.vflow is the hand-off.

1

Author & repair

in VL Agent

2

Govern

via Resource Center (review-gated)

3

Import

into VLCode — generate, run, adjust

> ***VL Agent produces, governs, validates. VLCode consumes. .vflow is the contract between them.***

10 WHY .VFLOW IS STRONG

One self-describing, tamper-evident bundle.

Portable & plain

A ZIP: the complete Workflow JSON (a steps DAG, Spec 4.1), its subflows, an auto-generated resource manifest, prompts and docs — in plain folders, not a black box.

Declares its portability

self-contained / needs-runtime / needs-secrets — you know before you open it whether it will just run.

Tamper-evident

Every file is sha256-checksummed against the manifest; a mismatched or missing file is rejected on import. (Checksums, not signatures.)

Add an app block → Agent App

The same .vflow becomes installable; the flow's own human steps become its interaction cards. Per-node AI config travels inside the file.

11 WHICH ONE IS FOR ME?

Same foundation, two jobs.

VLCode

When you build complete enterprise applications — with backends, project managers, and long-step interaction. The productized harness for apps.

VL Agent

When you build the agents, tools, and automation that power those apps — runtime-first, installable, auditable, as a single .vflow.

One account, one site, one download — zero re-learning across products.

THE MOAT

Two products.
One foundation.
One .vflow.

When software scales and AI goes deeper, whoever controls the representation layer controls the ceiling.
And this is just the beginning.

github.com/VisualLogic-AI/VLCode-Lite-agent-os